

Parking Lot System

Full-stack mini-project — build a working parking lot in about three hours.

Stack: Node.js · React · SQL Time limit: ~3 hours Level: Beginner–Intermediate

The brief

You'll build a small but complete parking lot system with a database, a REST API, and a web front end. Vehicles enter, get a ticket, and are charged based on how long they stayed when they leave. The lot has a fixed number of slots for each vehicle type, so when a type is full the system should say so instead of issuing a ticket.

When a vehicle enters, the system checks whether a slot of that type is free. If one is, it records the entry time and issues a ticket. If not, it shows “Parking Full”. When the vehicle exits, the system records the exit time, works out how long it stayed, and charges the matching fare.

What you'll build

- Park a vehicle by entering its number and type, and receive a ticket.
- Exit a vehicle using either the ticket ID or the vehicle number.
- Show live slot availability for each vehicle type.
- Store the entry time and exit time for every vehicle.
- Calculate and display the fare when a vehicle exits.

Tech stack (mandatory)

- Backend: Node.js (Express recommended).
- Frontend: React.
- Database: any SQL database — MySQL or PostgreSQL are both fine.

Assumptions

- Fixed slots: Bike → 5, Car → 5, Truck → 2.
- Slot limits are fixed and defined in the backend code.
- Stay time is measured from entry to exit, rounded up to whole hours — any part of an hour counts as a full hour.

Pricing rules

Duration	Fare
Up to 3 hours	₹30
More than 3 and up to 6 hours	₹85
More than 6 hours	₹120

Reference UI

Your finished app should look and behave roughly like the dashboard below. It's a single screen showing slot availability, a form to park a vehicle, a form to exit a vehicle, the ticket that gets generated, and a list of currently parked vehicles. You don't need to match the styling exactly — focus on the behaviour and the information shown.

Parking lot management

7 of 12 slots occupied

Bike

3 / 5 free

Car

2 / 5 free

Truck

0 / 2 free

Full

Park a vehicle

Generates a ticket

Vehicle number

KA01AB1234

Vehicle type

Car

Generate ticket

Exit a vehicle

Calculates the fare

Ticket ID or vehicle number

TKT-1001

Exit and calculate fare

Duration

5 hours

Amount due

85

Sample ticket

TICKET ID	VEHICLE	TYPE	ENTRY TIME
TKT-1001	KA01AB1234	Car	10:30 AM

Currently parked

Ticket ID	Vehicle no.	Type	Entry time	Slot
TKT-1001	KA01AB1234	Car	10:30 AM	C-3
TKT-1002	TS09XY7788	Bike	11:05 AM	B-1
TKT-1003	MH12QP3344	Truck	09:15 AM	T-2

Reference dashboard — availability cards, park/exit forms, a sample ticket, and the live parked-vehicles list.

Database schema

A single table is enough. Don't store the available-slot counts as their own column — compute them by counting the rows that are still parked, so the numbers can never drift out of sync with reality.

```
CREATE TABLE tickets (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  ticket_id VARCHAR(20) UNIQUE NOT NULL, -- e.g. TKT-1001  
  vehicle_number VARCHAR(20) NOT NULL,  
  vehicle_type ENUM('bike','car','truck') NOT NULL,  
  entry_time DATETIME NOT NULL,  
  exit_time DATETIME DEFAULT NULL, -- NULL while parked  
  amount DECIMAL(6,2) DEFAULT NULL, -- filled in on exit  
  status ENUM('parked','exited') NOT NULL DEFAULT 'parked'
```

```
);
```

Keep the fixed slot limits in your backend code, not the database:

```
const LIMITS = { bike: 5, car: 5, truck: 2 };
```

To find how many slots of a type are in use, count the rows that are still parked and subtract from the limit:

```
SELECT COUNT(*) AS occupied FROM tickets
WHERE vehicle_type = 'car' AND status = 'parked';
```

Column	Type	What it's for
id	INT, primary key	Auto-increment row id
ticket_id	VARCHAR, unique	The ticket number shown to the user, e.g. TKT-1001
vehicle_number	VARCHAR	The number plate entered when parking
vehicle_type	ENUM(bike/car/truck)	Decides which slot pool and limit applies
entry_time	DATETIME	Set when the vehicle parks
exit_time	DATETIME, nullable	NULL while parked; set on exit
amount	DECIMAL, nullable	The fare, filled in on exit
status	ENUM(parked/exited)	'parked' by default; 'exited' after checkout

Finished early? Add a second `slots` table (`id`, `slot_type`, `slot_number`, `is_occupied`) with a foreign key from `tickets`, so the app can assign and display a real slot number (like C-3) instead of just a count.

API contract

Build these four endpoints. The exact request and response shapes are given so the frontend and backend agree from the start — mismatched field names are the most common time-sink.

Method	Endpoint	Purpose
GET	/api/slots	Current availability per vehicle type
POST	/api/park	Park a vehicle and return a ticket
POST	/api/exit	Exit a vehicle and return the fare
GET	/api/parked	List the currently parked vehicles

GET /api/slots

Returns the totals and free counts that power the three availability cards. Response (200):

```
{
  "bike": { "total": 5, "available": 3 },
  "car": { "total": 5, "available": 2 },
  "truck": { "total": 2, "available": 0 }
}
```

POST /api/park

Request body:

```
{ "vehicleNumber": "KA01AB1234", "vehicleType": "car" }
```

On success, return 201 with the new ticket:

```
{
  "success": true,
  "ticket": {
    "ticketId": "TKT-1001",
    "vehicleNumber": "KA01AB1234",
    "vehicleType": "car",
    "entryTime": "2026-07-07T10:30:00Z"
  }
}
```

When that vehicle type is full, return 409:

```
{ "success": false, "message": "Parking Full" }
```

POST /api/exit

Accepts either identifier — the ticket ID or the vehicle number:

```
{ "ticketId": "TKT-1001" }

{ "vehicleNumber": "KA01AB1234" }
```

On success, return 200 with the bill:

```
{
  "success": true,
  "receipt": {
    "ticketId": "TKT-1001",
    "vehicleNumber": "KA01AB1234",
    "entryTime": "2026-07-07T10:30:00Z",
    "exitTime": "2026-07-07T14:45:00Z",
    "durationHours": 5,
    "amount": 85
  }
}
```

```
}  
}
```

If no matching parked vehicle exists, return 404:

```
{ "success": false, "message": "Ticket not found or already exited" }
```

GET /api/parked

Returns an array of the currently parked vehicles for the table:

```
[  
  {  
    "ticketId": "TKT-1001",  
    "vehicleNumber": "KA01AB1234",  
    "vehicleType": "car",  
    "entryTime": "2026-07-07T10:30:00Z"  
  }  
]
```

Edge cases to handle. Return a 400 with a clear message rather than crashing when: the same vehicle number tries to park while it's already parked, the vehicle type is missing or invalid, or the request body is empty.

Pricing logic

The slabs overlap at the boundaries (3 and 6 each appear twice), so pick a convention and stick to it. The cleanest is to round the stay up to whole hours, which removes the ambiguity entirely.

```
function calculateFare(entryTime, exitTime) {  
  const ms = new Date(exitTime) - new Date(entryTime);  
  const hours = Math.ceil(ms / (1000 * 60 * 60)); // round up to full hours  
  
  if (hours <= 3) return 30;    // up to 3 hours  
  if (hours <= 6) return 85;    // 3 to 6 hours  
  return 120;                  // more than 6 hours  
}
```

So 1–3 hours is ₹30, 4–6 hours is ₹85, and 7+ hours is ₹120. Always compute the fare on the server from the stored entry and exit times — never trust a duration sent from the browser, since that's trivial to fake.

Suggested build order

The usual mistake is building the whole frontend before the backend works, then debugging both at once. Go one layer at a time and test each with Postman or curl before moving up. Rough timings for the three-hour budget are in brackets.

1. Database first — create the table and insert one row by hand to confirm the connection works. (~30 min)
2. Backend endpoints, one at a time, in order: slots → park → exit → parked. Test each before writing the next. (~60 min)
3. React frontend — availability cards that load on start, then the park and exit forms wired to their endpoints. (~75 min)
4. Error states and polish — especially the “Parking Full” message and the fare display. (~15 min)

Suggested packages: express, cors, and mysql2 (or pg for PostgreSQL) on the backend; plain fetch or axios on the frontend. Nothing heavier is needed at this scope.

Submission checklist

Before you submit, make sure you have:

- ☐ A repository (or zip) containing both the backend and the frontend.
- ☐ The SQL schema file included, plus setup steps in a short README.
- ☐ All four API endpoints working.
- ☐ Park, exit, availability, and the parked list all functional.
- ☐ “Parking Full” shown when a type is full, and invalid input handled without crashing.

Grading rubric

Criterion	Points
Park works and slot availability updates correctly	25
Exit works and the fare is calculated correctly	25
Data persisted in SQL, including entry and exit times	20
UI shows availability, tickets, and the parked list clearly	15
Handles “Parking Full” and invalid input gracefully	10
Code organisation and a working README	5
Total	100